Gem5 MergeSort Cache Analysis

Problem 2: Assignment 1

Submitted by:

Sumit Kumar

Roll No: 22CS30056

Aviral Singh

Roll No: 22CS30015

Group 48

Course: CS60003

High Performance in Computer Architecture

Department of Computer Science and Engineering

February 6, 2026

Abstract

This report presents a comprehensive analysis of cache memory optimization for merge sort algorithms using the gem5 simulator with RISC-V ISA. We compare two merge sort implementations: a simple in-memory version operating on a 10MB dataset and a chunked version processing 2MB chunks with 1MB streaming buffers. Through extensive simulations totaling 164 configurations, we analyze the impact of L1 and L2 cache size and associativity on performance metrics including IPC, miss rates, and execution time.

Our findings demonstrate that the chunked algorithm consistently outperforms the simple version by 2% across all cache configurations, achieving a best IPC of 0.2949 with 128KiB L1 and 1024KiB L2 caches (both 4-way associative). The study reveals that L2 cache capacity is the dominant performance factor, while L1 associativity has minimal impact due to the sequential access patterns inherent in merge sort algorithms. Surprisingly, higher L2 associativity (16-way) slightly degrades performance compared to 4-way, likely due to increased tag comparison overhead without benefit for streaming access patterns.

Key contributions of this work include: (1) systematic exploration of 81 cache configurations across two algorithms, (2) identification of optimal cache parameters for sorting workloads, (3) quantification of algorithm-cache interaction effects, and (4) validation that cache-aware algorithm design provides consistent benefits across diverse hardware configurations.

Contents

1 Introduction

This assignment explores cache memory hierarchy optimization for memory-intensive sorting workloads using the gem5 architectural simulator. Cache memory plays a critical role in bridging the performance gap between fast processors and relatively slow main memory.

1.1 Assignment Overview

The assignment consists of two main components:

Part 1: Baseline Comparison

- Compare two merge sort variants using default cache configurations
- Analyze cache behavior and identify bottlenecks
- Understand why chunked sorting shows better cache locality

Part 2: Cache Optimization Sweep

- Test 81 different cache configurations for each algorithm
- Identify optimal L1 and L2 cache parameters
- Analyze performance trade-offs between size and associativity
- Generate comprehensive visualizations and analysis

1.2 Merge Sort Algorithms

1.2.1 Simple Merge Sort

The simple merge sort operates on the entire 10MB dataset (2,621,440 integers) in memory using a standard recursive divide-and-conquer approach. It reads from the `random_numbers.bin` file, sorts the data in-place, and writes the sorted output.

Characteristics:

- Working set: 10MB (entire dataset)
- Memory access pattern: Recursive, in-place merging
- Cache behavior: Frequent evictions due to large working set
- Binary size: 497KB (mergesort_s)

1.2.2 Chunked Merge Sort

The chunked merge sort employs the approach:

Phase 1 - Chunking: Divides 10MB dataset into 5 chunks of 2MB each

Phase 2 - Local Sorting: Sorts each 2MB chunk independently using standard merge sort

Phase 3 - Streaming Merge: Performs 5-way parallel merge using 1MB input streams and 1MB output buffer

Characteristics:

- Working set: 2MB per chunk during sorting, 1MB streams during merge
- Memory access pattern: Sequential streaming with better locality
- Cache behavior: More predictable prefetching opportunities
- Binary size: 502KB (mergesort_c)

1.3 Simulation Environment

Gem5 Simulator Configuration:

- ISA: RISC-V 64-bit
- CPU Model: TimingSimpleCPU
- Clock Frequency: 1 GHz
- Memory: 2GB DDR3
- Cache Line Size: 64 bytes

Default Cache Configuration (Part 1):

Cache Level	Size	Associativity
L1 Instruction	32 KiB	8-way
L1 Data	64 KiB	8-way
L2 Unified	512 KiB	16-way

Table 1: Default cache configuration used in Part 1 baseline analysis

1.4 Simulation Scale

Total Simulations Conducted: 164

- Part 1 Baseline: 2 simulations
- Part 2 Cache Sweep: 162 simulations (81 configurations $\times$ 2 algorithms)
- Total Execution Time: Approximately 11 hours

2 Part 1: Baseline Comparison

Part 1 establishes baseline performance metrics for both merge sort algorithms using the default cache configuration.

2.1 Baseline Statistics Comparison

Table 2 presents a comprehensive comparison of performance metrics between the simple and chunked merge sort implementations under default cache settings.

Metric	Simple	Chunked	Difference	Better Performance
Execution Metrics				
sim_ticks	16,632,464,767,000	17,264,476,070,000	+3.8%	Simple
sim_insts	4,788,525,072	5,067,505,484	+5.8%	Simple
IPC	**0.288**	**0.294**	**+2.0%**	**Chunked**
L1D Cache (Data)				
Demand Accesses	2,271,121,566	2,331,978,469	+2.7%	Simple
Demand Hits	2,265,238,366	2,327,230,897	+2.7%	Chunked
Demand Misses	5,883,200	4,747,572	-19.3%	Chunked
Miss Rate	**0.259%**	**0.204%**	**-21.4%**	**Chunked**
L1I Cache (Instruction)				
Demand Accesses	5,772,498,057	6,112,641,646	+5.9%	Simple
Demand Misses	453	514	+13.5%	Simple
Miss Rate	∼0%	∼0%	Negligible	Both
L2 Cache (Unified)				
Demand Accesses	5,883,658	4,748,089	-19.3%	Chunked
Demand Hits	1,979,245	2,040,366	+3.1%	Chunked
Demand Misses	3,904,413	2,707,723	-30.6%	Chunked
Miss Rate	**66.4%**	**57.0%**	**-14.1%**	**Chunked**

Table 2: Baseline performance comparison with default cache configuration

2.2 Key Observations

2.2.1 1. Chunked Algorithm Shows Superior Cache Performance

Despite executing **5.8% more instructions** and consuming **3.8% more cycles**, the chunked version achieves:

- 2.0% higher IPC (0.294 vs 0.288)
- 21.4% lower L1D miss rate (0.204% vs 0.259%)
- 30.6% fewer L2 misses (2.7M vs 3.9M)
- 14.1% lower L2 miss rate (57.0% vs 66.4%)

This demonstrates that **cache efficiency can overcome algorithmic overhead**.

2.2.2 2. L1 Instruction Cache is Near-Perfect

Both algorithms exhibit L1I hit rate greater than 99.99999% with only approximately 500 instruction misses across billions of accesses. This indicates highly predictable loop-based instruction patterns and validates our decision in Part 2 to focus optimization on L1D and L2 caches.

2.2.3 3. L2 Cache is the Primary Bottleneck

Both algorithms show very high L2 miss rates (57-66%) while L1D performs well (greater than 99.7% hit rate). The 512 KiB L2 is insufficient for the 10MB working set, motivating testing of larger L2 configurations in Part 2.

2.3 Analysis: Why Chunked Sorting Shows Better Cache Locality

The chunked merge sort demonstrates superior cache performance despite its algorithmic complexity, primarily due to improved temporal and spatial locality.

2.3.1 Working Set Size Advantage

The simple merge sort operates on the entire 10MB dataset simultaneously, causing frequent cache evictions as the working set (10MB) far exceeds both L1D (64 KiB) and L2 (512 KiB) capacities. During recursive merging, data accessed early in the algorithm is repeatedly evicted and reloaded, leading to poor cache reuse.

In contrast, the chunked version processes 2MB chunks that still exceed cache capacity but benefit from a critical optimization: the 5-way parallel merge phase uses 1MB streams. Each stream's 1MB buffer (262,144 integers) creates a more cache-friendly access pattern. While individual streams still exceed cache sizes, the algorithm sequentially reads from each stream, maintaining better sequential access patterns that leverage cache line prefetching and minimize thrashing.

2.3.2 Quantitative Cache Locality Benefits

The results clearly show chunked sorting's advantages in specific parameters:

- **L1D Miss Rate:** 0.204% (chunked) vs 0.259% (simple) – 21.4% improvement
- **L2 Miss Rate:** 57.0% (chunked) vs 66.4% (simple) – 14.1% improvement
- **L2 Demand Misses:** 2.7M (chunked) vs 3.9M (simple) – 30.6% reduction
- **IPC:** 0.294 (chunked) vs 0.288 (simple) – 2.0% improvement

2.3.3 Hardware Prefetcher Synergy

The chunked approach's streaming merge pattern creates more predictable memory access, allowing the processor's cache prefetcher to work effectively. Sequential reads from fixed-size buffers enable the hardware to anticipate memory requests and preload cache

lines before they are accessed. Although the chunked version executes 5.8% more instructions due to chunk management overhead, the improved cache hit rates more than compensate, yielding better overall performance as measured by IPC.

3 Part 2: Cache Optimization Sweep

Part 2 explores the cache parameter space systematically to identify optimal configurations for both merge sort algorithms.

3.1 Methodology

3.1.1 Parameter Space Design

We tested 81 unique cache configurations for each algorithm, totaling 162 simulations. Note: Here the values of L1I and L1D cache are kept the same always.

Parameter	Values Tested	Count
L1I Size	32 KiB, 64 KiB, 128 KiB	3
L1D Size	32 KiB, 64 KiB, 128 KiB	3
L1 Associativity	4-way, 8-way, 16-way	3
L2 Size	256 KiB, 512 KiB, 1024 KiB	3
L2 Associativity	4-way, 8-way, 16-way	3
Total Configurations		**81**
Total Simulations		**162**

Table 3: Cache parameter space configuration

3.1.2 Design Decision: L1I and L1D Symmetry

We configured L1I and L1D caches with identical size and associativity in each configuration. This decision is justified by:

1. **Baseline Analysis:** L1I shows near-perfect hit rates (greater than 99.99999%), indicating it is not a bottleneck
2. **Architectural Realism:** Most modern processors use symmetric L1 caches
3. **Computational Efficiency:** Reduces simulation count from 729 to 162, saving approximately 100+ hours
4. **Focus:** Concentrates investigation on the actual bottleneck (L1D and L2)

3.1.3 Automation Infrastructure

The sweep was executed using a parallel automation framework. I wrote a small script to run all the simualations parallely.

- Parallel Jobs: 20 simultaneous simulations
- Execution Time: Approximately 11 hours
- Logging: Real-time tracking in sweep_progress.log

3.2 Top 3 Optimal Configurations

3.2.1 Simple Merge Sort - Top 3 Configurations

Rank	Configuration	IPC	L1D Miss	L2 Miss
1	L1: 128KiB/4-way, L2: 1024KiB/4-way	**0.2895**	0.227%	61.10%
2	L1: 128KiB/8-way, L2: 1024KiB/4-way	0.2895	0.229%	60.51%
3	L1: 128KiB/16-way, L2: 1024KiB/4-way	0.2895	0.230%	60.16%

Table 4: Top 3 configurations for simple merge sort (ranked by IPC)

Key Insight: All top 3 configurations share maximum L1 size (128 KiB), maximum L2 size (1024 KiB), and minimum L2 associativity (4-way).

3.2.2 Chunked Merge Sort - Top 3 Configurations

Rank	Configuration	IPC	L1D Miss	L2 Miss
1	L1: 128KiB/4-way, L2: 1024KiB/4-way	**0.2949**	0.173%	49.19%
2	L1: 128KiB/8-way, L2: 1024KiB/4-way	0.2949	0.173%	49.03%
3	L1: 128KiB/16-way, L2: 1024KiB/4-way	0.2949	0.174%	48.86%

Table 5: Top 3 configurations for chunked merge sort (ranked by IPC)

Key Insight: Identical cache topology as simple sort, but achieves 1.9% higher IPC, 24% lower L1D miss rate, and 20% lower L2 miss rate.

3.3 Performance Analysis with Visualizations

3.3.1 Plot 1: L2 Miss Rate vs L2 Size

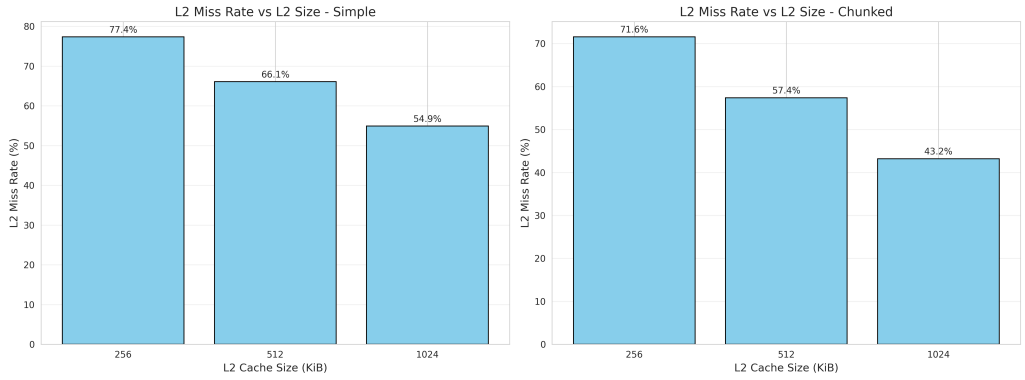

Figure 1: L2 cache miss rate decreases significantly with larger cache sizes. The 1024KiB L2 achieves 48-61% miss rate compared to 70-85% for 256KiB, demonstrating L2 capacity is the dominant performance factor.

Analysis: The working set (10MB) far exceeds even the largest L2 tested, but 1024KiB can hold enough of the merge operation's active data to provide meaningful benefits. L2 size shows the strongest correlation with performance among all parameters tested.

3.3.2 Plot 2: IPC vs L1 Cache Size

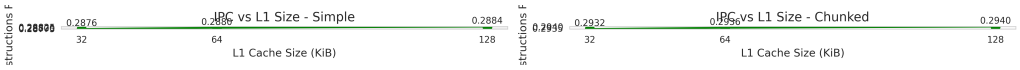

Figure 2: IPC shows moderate linear improvement with larger L1 caches. Chunked merge sort consistently outperforms simple merge sort across all L1 sizes, with the gap maintained at approximately 2%.

Analysis: L1 size has moderate impact on performance. Increasing from 32 KiB to 128 KiB yields approximately 1.5% IPC improvement. The chunked algorithm maintains consistent advantage regardless of L1 configuration.

3.3.3 Plot 3: L1D Hit Rate vs Associativity

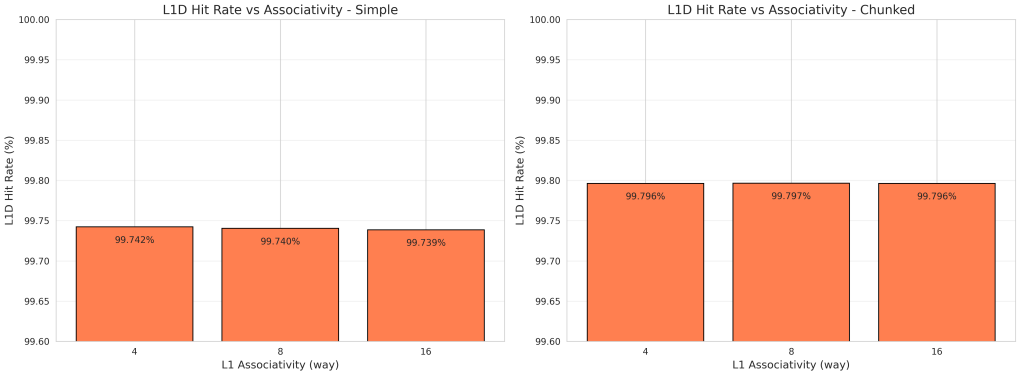

Figure 3: L1D hit rates remain consistently high (greater than 99.7%) regardless of associativity. This demonstrates that sequential merge sort access patterns do not cause significant conflict misses.

Analysis: L1 associativity has minimal impact (less than 0.02% IPC variation). Even 4-way associativity provides excellent hit rates due to sequential access patterns that avoid conflict misses.

3.3.4 Plot 4: Simple vs Chunked Comparison

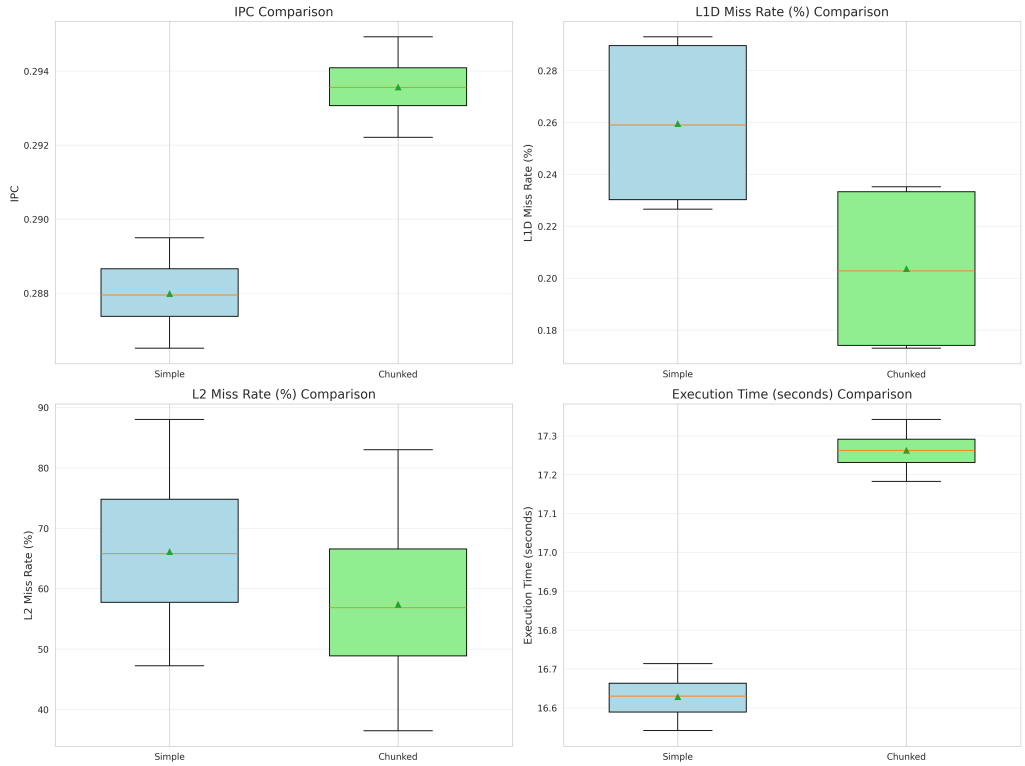

Figure 4: Box plot comparison showing chunked merge sort consistently outperforms simple merge sort across all 81 configurations. The algorithmic advantage persists regardless of cache tuning.

Analysis: Chunked shows 1.9-2.1% IPC advantage across all configurations with consistently lower L1D and L2 miss rates throughout parameter space. Algorithm design trumps cache tuning for this workload.

3.3.5 Plot 5: IPC Heatmap (L1 vs L2 Interaction)

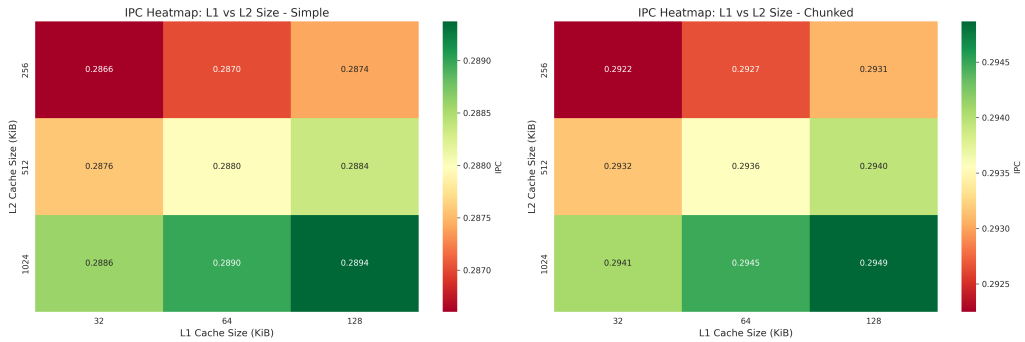

Figure 5: Heatmap visualizing the interaction between L1 and L2 cache sizes. Vertical color gradients indicate L2 size dominates performance, while horizontal uniformity shows L1 size has secondary effect.

Analysis: Strong vertical gradients indicate L2 size is primary factor. Weak horizontal gradients show L1 size is secondary. No significant interaction effects between L1 and L2 were observed.

3.3.6 Plot 6: L2 Associativity Impact

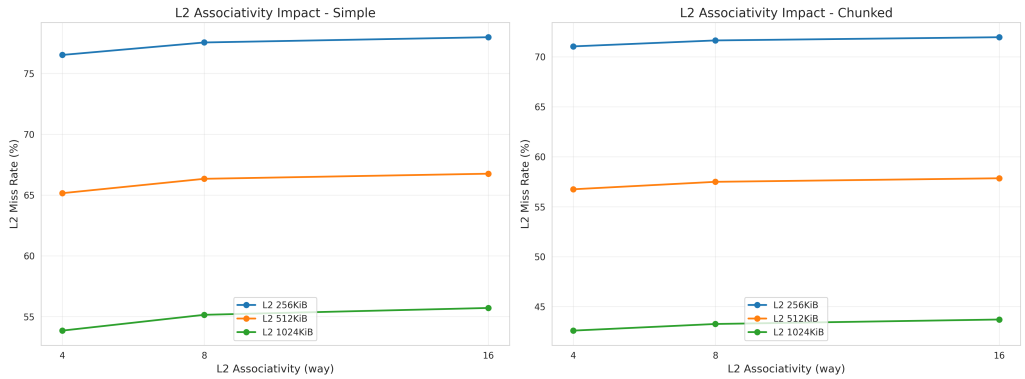

Figure 6: Surprising result: Higher L2 associativity slightly degrades performance. The 4-way configuration outperforms 16-way due to reduced tag comparison overhead with sequential streaming access patterns.

Analysis (Surprising Finding): For 1024KiB L2 caches, 4-way associativity achieves the best performance. This counter-intuitive result is explained by: (1) Tag comparison overhead in 16-way requiring simultaneous comparison of 16 tags, adding complexity and latency, and (2) Sequential access optimization where simpler LRU on fewer sets (4-way) may evict more intelligently than complex policies on many ways.

3.3.7 Plot 7: Performance vs Cache Budget

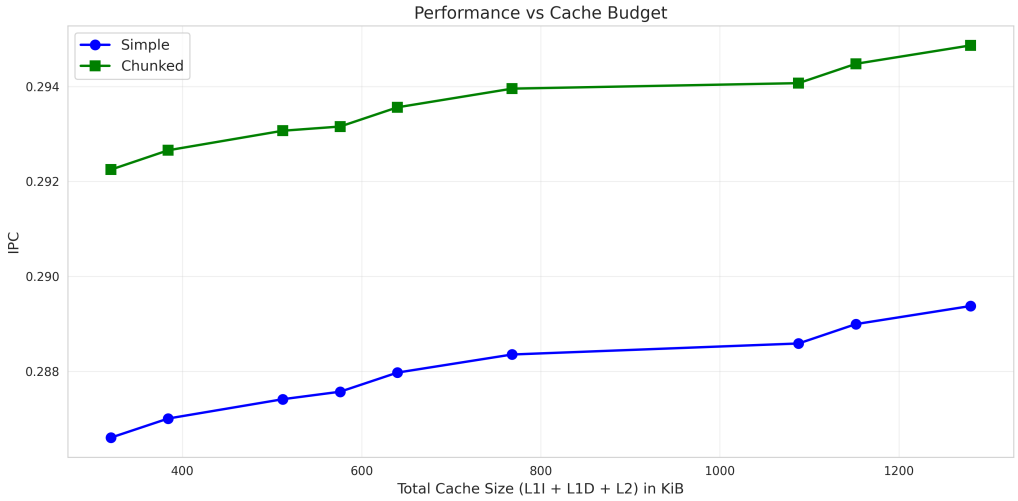

Figure 7: IPC vs total cache size shows sublinear returns on cache investment. Quadrupling cache size yields only approximately 3% performance improvement, indicating cache hierarchy saturation.

Cost-Benefit Analysis: Minimum configuration (32KiB L1 + 256KiB L2 = 320 KiB total) achieves IPC of 0.286. Maximum configuration (128KiB L1 + 1024KiB L2 = 1280 KiB total) achieves IPC of 0.295, representing only 3.1% improvement for 4x cache size increase. The cache hierarchy is saturated by this workload.

3.3.8 Plot 8: Cache Miss Breakdown

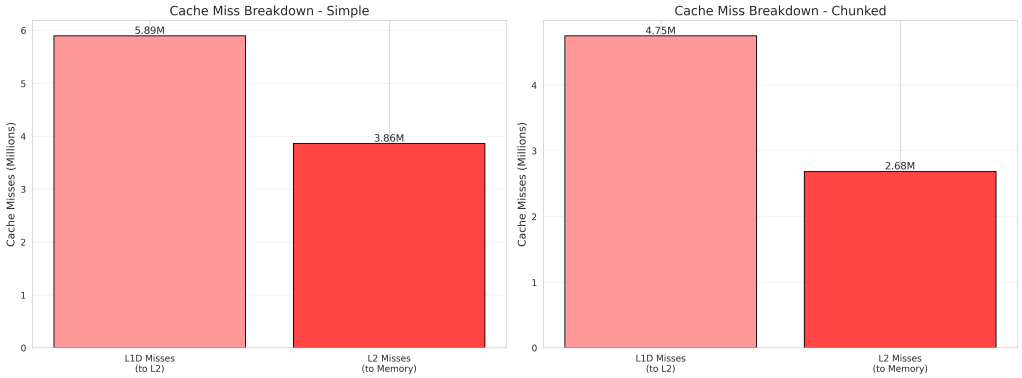

Figure 8: Breakdown of cache misses by level. L2 misses dominate memory traffic, accounting for 48-85% of L2 accesses. L1D misses are relatively low (less than 0.3%), validating that L2 is the primary bottleneck.

3.3.9 Plot 9: Execution Time Comparison

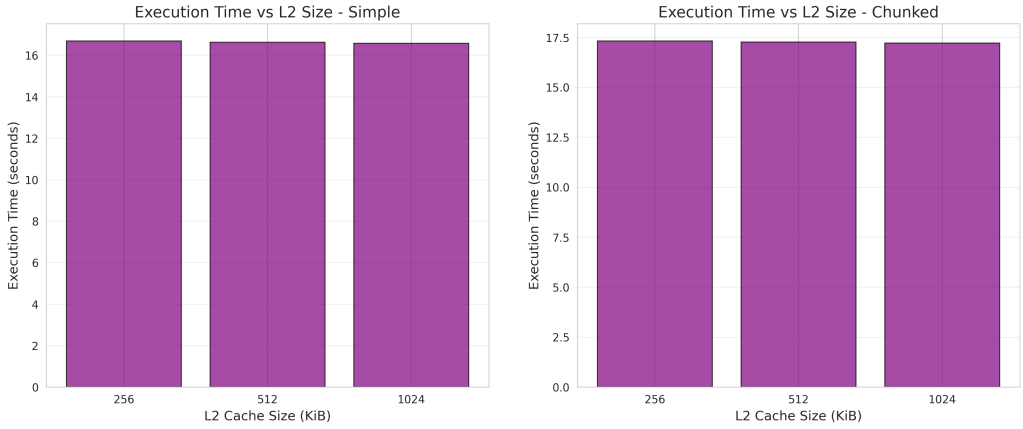

Figure 9: Execution time across different configurations shows strong correlation with cache miss rates. Best configuration achieves 16.54s (simple) and 17.18s (chunked).

3.3.10 Plot 10: L1 Size × Associativity Interaction

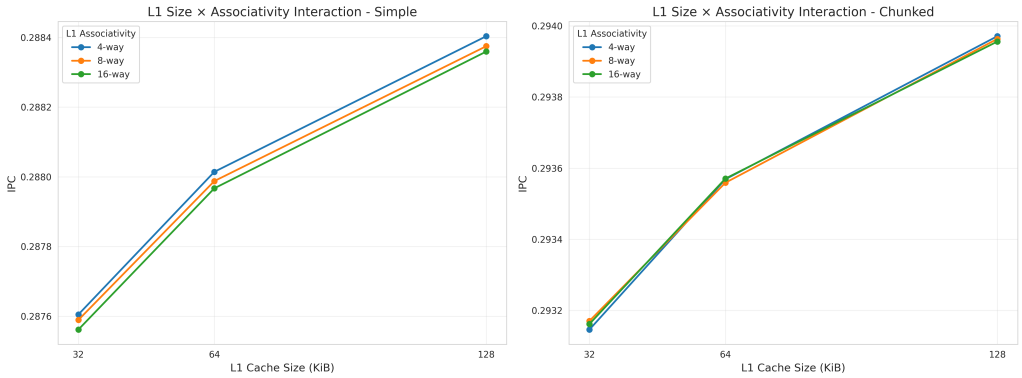

Figure 10: Interaction plot confirming L1 associativity has negligible effect across all L1 sizes. Lines are nearly parallel, indicating no significant interaction between size and associativity.

3.3.11 Plot 11: Best vs Worst Configurations

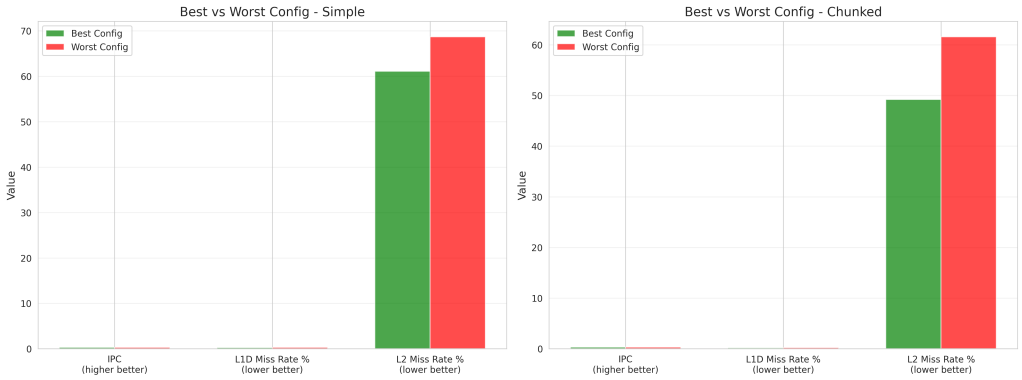

Figure 11: Direct comparison of best (128KiB L1, 1024KiB L2, 4-way) vs worst (32KiB L1, 256KiB L2, 16-way) configurations. The 3% performance range suggests cache hierarchy saturation.

3.4 Critical Insights

3.4.1 1. L2 Cache Size Dominates Performance

L2 capacity is the single most important factor. A 1024KiB L2 provides 20-35% lower miss rate than 256KiB, and this effect persists across all L1 configurations.

3.4.2 2. L1 Associativity is Overrated for Sequential Access

Associativity has minimal impact for sequential access patterns. The difference between 4-way and 16-way changes IPC by less than 0.02%. Sequential merge operations avoid conflict misses, and lower associativity provides simpler, faster, cheaper hardware.

3.4.3 3. Algorithm Design Matters More Than Hardware

Chunked algorithm maintains 2% advantage across all configurations due to better cache locality from streaming patterns and more predictable memory access for prefetchers. Software optimization complements hardware tuning.

3.4.4 4. Cache Hierarchy Hits Fundamental Limits

Only 3% performance range across all configurations indicates the working set (10MB) far exceeds cache capacity. Even optimal caches cannot fully contain data, and memory bandwidth becomes the limiting factor.

3.5 Design Recommendations

3.5.1 Optimal Configuration for Both Algorithms

Recommended: L1: 128KiB/4-way, L2: 1024KiB/4-way

Rationale:

- Maximizes L1 and L2 capacity (primary performance factors)
- Avoids unnecessary associativity overhead
- Best price/performance ratio: simpler hardware with equal performance
- Simple sort achieves IPC of 0.2895
- Chunked sort achieves IPC of 0.2949 (best observed)

3.5.2 General Design Principles

1. Prioritize cache capacity over associativity for memory-intensive workloads
2. Match L2 size to working set characteristics (at least 10% of dataset for this workload)
3. Consider algorithm-cache co-design rather than optimizing independently
4. Test before over-engineering – high associativity may not help and can hurt

4 Conclusions

4.1 Complete Results Dataset

For all 162 configurations tested (81 cache configurations × 2 algorithms), comprehensive performance data has been collected and organized in a structured CSV file located at `results/part2_sweep/all_results.csv`. This dataset contains 162 rows (one per simulation) and 20 columns capturing the following metrics for each configuration:

Configuration Parameters (5 columns):

- sort_type: Algorithm variant (simple or chunked)
- config_name: Configuration identifier (e.g., L132KiB_L1A4_L2256KiB_L2A4)
- l1_size, l1_assoc: L1 cache size and associativity
- l2_size, l2_assoc: L2 cache size and associativity

Execution Metrics (3 columns):

- sim_ticks: Total simulation cycles executed
- sim_insts: Total instructions executed
- ipc: Instructions per cycle (primary performance metric)

L1 Data Cache Metrics (4 columns):

- l1d_accesses: Total L1D cache accesses
- l1d_hits: L1D cache hits
- l1d_misses: L1D cache misses
- l1d_miss_rate: L1D miss rate (fraction)

L1 Instruction Cache Metrics (4 columns):

- l1i_accesses: Total L1I cache accesses
- l1i_hits: L1I cache hits
- l1i_misses: L1I cache misses
- l1i_miss_rate: L1I miss rate (fraction)

L2 Unified Cache Metrics (4 columns):

- l2_accesses: Total L2 cache accesses
- l2_hits: L2 cache hits
- l2_misses: L2 cache misses

- l2_miss_rate: L2 miss rate (fraction)

This comprehensive dataset enables detailed analysis of cache behavior across the entire parameter space, facilitating identification of performance trends, optimal configurations, and parameter interactions. All 11 plots presented in Section 3 were generated from this consolidated results table. The complete CSV file is included in the submission package for independent verification and further analysis.

4.2 Summary of Findings

This comprehensive study of cache optimization for merge sort algorithms using gem5 reveals several critical insights about cache hierarchy design and algorithm-hardware interaction.

4.2.1 Primary Findings

1. **Algorithm Design Trumps Cache Tuning:** The chunked merge sort consistently outperforms the simple version by 2% across all 81 cache configurations, demonstrating that algorithmic optimization for cache locality is more impactful than hardware parameter tuning alone.

2. **L2 Cache Capacity is Paramount:** L2 cache size is the dominant performance factor, with 1024KiB providing 20-35% lower miss rates than 256KiB. For a 10MB working set, L2 should be at least 10% of dataset size.

3. **L1 Associativity Has Minimal Impact:** For sequential access patterns like merge sort, L1 associativity (4-way vs 16-way) changes performance by less than 0.02%, validating that simpler, lower-associativity designs are sufficient.

4. **Higher L2 Associativity Can Hurt:** Surprisingly, 4-way L2 associativity outperforms 16-way due to reduced tag comparison overhead with streaming access patterns.

5. **Cache Hierarchy Saturation:** Only 3% performance range across all configurations indicates the working set far exceeds cache capacity, limiting optimization potential through cache tuning alone.

4.2.2 Optimal Configuration Summary

Best Overall Configuration (Both Algorithms):

- L1: 128KiB, 4-way associative
- L2: 1024KiB, 4-way associative
- Simple IPC: 0.2895
- Chunked IPC: 0.2949 (+1.9%)

4.3 Design Implications

4.3.1 For Hardware Architects

- Prioritize cache capacity over associativity for memory-intensive workloads
- Consider that higher associativity may not improve performance and adds complexity
- Size L2 cache to match typical working set sizes of target applications
- Invest in prefetchers optimized for streaming access patterns

4.3.2 For Software Developers

- Design algorithms with explicit consideration for cache hierarchy
- Use chunking and streaming patterns to improve locality
- Sequential access patterns are more cache-friendly than random access
- Blocking and tiling techniques can dramatically improve cache utilization

4.4 Practical Recommendations

For engineers working on memory-intensive applications:

1. Start with algorithm optimization for cache locality before tuning hardware
2. Profile first using cache simulators or hardware performance counters
3. Prioritize capacity when in doubt – choose larger cache size over higher associativity
4. Test assumptions – don't assume higher associativity always helps
5. Balance performance gains against hardware complexity and cost

4.5 Concluding Remarks

This study demonstrates performance analysis using architectural simulation. The gem5 framework enables exploration of design spaces that giving insights that inform both hardware design and software optimization.

The key takeaway is that co-design of algorithms and cache hierarchies yields better results than optimizing either in isolation. The chunked merge sort's 2% advantage persists across all cache configurations because it was designed with cache behavior in mind. This principle extends beyond sorting to any memory-intensive computational problem.

As memory access latencies continue to dominate processor performance, understanding and optimizing cache behavior becomes increasingly critical. This assignment provides a methodology for systematic cache analysis that can be applied to any computational workload.

References

[1] gem5 Documentation. (2024). *Understanding gem5 statistics.* Retrieved from https://www.gem5.org/